

Atividade 17 – Revisão da prova

Estudo Guiado de Revisão – SQL e CRUD

Contexto: Sistema de Reservas de Hotel

Objetivo

Este estudo guiado reúne os conteúdos trabalhados nas atividades anteriores de SQL e CRUD, agora utilizando um **sistema de reservas de hotel** como contexto único para toda a revisão.

Ao todo, a lista possui 50 exercícios, distribuídos entre:

- consultas básicas;
- seleção e expressões lógicas;
- tratamento de NULL;
- LIKE, UPPER e LOWER;
- consultas com múltiplas tabelas;
- INNER JOIN, LEFT JOIN e RIGHT JOIN;
- aliases;
- expressões calculadas;
- DISTINCT;
- GROUP BY;
- COUNT, SUM e AVG;
- HAVING;
- ORDER BY;
- UNION, UNION ALL e INTERSECT;
- subconsultas;
- CRUD com PostgreSQL;
- integridade referencial.

Utilize o arquivo:

banco_hotel_reservas_postgresql.sql

O banco principal utilizado na atividade possui as tabelas:

tipo_quarto

quarto

hospede

reserva

funcionario

Os principais relacionamentos são:

tipo_quarto 1 ---- N quarto

hospede 1 ---- N reserva

quarto 1 ---- N reserva

funcionario 1 ---- N funcionario supervisor

A tabela funcionario possui um autorrelacionamento: um funcionário pode possuir outro funcionário como supervisor.

O objetivo desta revisão não é apenas memorizar sintaxe. Antes de escrever SQL, identifique **quais dados precisam aparecer, de quais tabelas eles vêm, como essas tabelas se relacionam e quais condições determinam o resultado.**

Parte I – SELECT, projeção, seleção e expressões lógicas

1. Contextualização

O comando SELECT é utilizado para consultar dados:

Sua forma básica é:

```
SELECT coluna1, coluna2  
FROM tabela;
```

Para recuperar todas as colunas:

```
SELECT *  
FROM tabela;
```

A escolha das colunas que aparecerão no resultado é chamada de **projeção**.

Por exemplo:

```
SELECT numero, andar, status  
FROM quarto;
```

A tabela quarto possui outras informações, mas apenas essas três colunas aparecem no resultado.

2. Seleção com WHERE

A cláusula WHERE determina quais linhas permanecem no resultado.

```
SELECT *  
FROM quarto  
WHERE status = 'Disponível';
```

Podemos utilizar operadores relacionais:

```
=  
<>  
!=  
>  
<  
>=  
<=
```

e operadores lógicos:

```
AND  
OR  
NOT
```

Exemplo:

```
SELECT *  
FROM tipo_quarto  
WHERE capacidade >= 3  
      AND valor_diaria < 600;
```

Com OR:

```
WHERE status = 'Disponível'  
      OR status = 'Manutenção'
```

Parênteses ajudam a explicitar a ordem das condições:

```
WHERE andar >= 2  
      AND (status = 'Disponível' OR status = 'Ocupado')
```

3. Valores NULL

NULL representa ausência de valor.

No banco do hotel, alguns hóspedes não possuem telefone informado e alguns quartos não possuem informação de visita.

Não devemos escrever:
telefone = NULL

O correto é:
telefone IS NULL

Ou:
telefone IS NOT NULL

4. UPPER e LOWER

Podemos normalizar temporariamente uma cadeia de caracteres:

```
SELECT *  
FROM hospede  
WHERE LOWER(cidade) = 'curitiba';
```

Assim, valores como Curitiba e CURITIBA podem ser encontrados pela mesma consulta.

5. LIKE

LIKE permite comparar textos com padrões.

% → qualquer sequência de caracteres

_ → exatamente um caractere

Exemplos:

Nomes iniciados por A:

WHERE nome LIKE 'A%'

Nome contendo a:

WHERE nome LIKE '%a%'

Número de quarto com exatamente três caracteres:

WHERE numero LIKE '___'

Exercícios – Parte I

Exercício 1

Liste o identificador, o número, o andar e o status de todos os quartos.

Resposta: SELECT id_quarto, numero, andar, status
FROM quarto;

Exercício 2

Liste os hóspedes da cidade de Curitiba.

Apresente:

- id;
- nome;
- cidade.

Resposta: SELECT id_hospede, nome, cidade
FROM hospede
WHERE cidade = 'Curitiba';

Exercício 3

Liste os tipos de quarto cuja diária seja superior a 300.00.

Apresente:

- nome do tipo;
- valor da diária;
- capacidade.

Resposta: SELECT nome_tipo, valor_diaria, capacidade
FROM tipo_quarto
WHERE valor_diaria > 300.00;

Exercício 4

Liste os tipos de quarto que:

- tenham capacidade para pelo menos 2 pessoas;
- possuam diária maior ou igual a 400.00.

Resposta: SELECT *

```
FROM tipo_quarto
WHERE capacidade >= 2
      AND valor_diaria >= 400.00;
```

Exercício 5

Liste quartos cujos status seja Disponível ou Ocupado, mas apenas aqueles localizados a partir do terceiro andar.
Utilize AND, OR e parênteses.

Resposta: SELECT *
FROM quarto
WHERE andar >= 3
 AND (status = 'Disponível' OR status = 'Ocupado');

Exercício 6

Liste os hóspedes que não possuem telefone informado.
Depois, escreva outra consulta que liste somente hóspedes com telefone informado.

Resposta:
-- Hóspedes sem telefone
SELECT *
FROM hospede
WHERE telefone IS NULL;

-- Hóspedes com telefone
SELECT *
FROM hospede
WHERE telefone IS NOT NULL;

Exercício 7

Liste hóspedes de Curitiba, independentemente de a cidade estar armazenada com letra maiúsculas ou minúsculas.

Utilize LOWER ou UPPER.

Resposta: SELECT *
FROM hospede
WHERE LOWER (cidade) = 'curitiba';

Exercício 8

Liste hóspedes cujo nome começa com a letra A.
Apresente id, nome e cidade.

Resposta: SELECT id_hospede, nome, cidade
FROM hospede
WHERE nome LIKE 'A%'

Exercício 9

Liste tipos de quarto cujo nome contenha a letra a.

Depois, modifique a consulta para buscar hóspedes cujo nome termine com a.

Resposta:

-- Tipos de quarto com 'a'

```
SELECT *  
FROM tipo_quarto  
WHERE LOWER (nome_tipo) LIKE '%a%';
```

-- Hóspedes cujo nome termina com 'a'

```
SELECT *  
FROM hospede  
WHERE nome LIKE '%a';
```

Exercício 10

Liste reservas que:

- estejam com status Confirmada ou Concluída;
- possuam pelo menos 2 hóspedes;
- tenham valor total superior a 500.00.

Resposta: SELECT *

```
FROM reserva  
WHERE (status = 'Confirmada' OR status = 'Concluída')  
      AND quantidade_hospedes >= 2  
      AND valor_total > 500.00;
```

Parte II – Consultas com múltiplas tabelas e JOIN

6. Por que combinar tabelas?

Em um banco relacional, informações relacionadas ficam separadas.

A tabela quarto, por exemplo, possui:

id_tipo

mas o nome da categoria está em:

tipo_quarto

Para obter:

301 | Luxo

precisamos combinar as tabelas.

7. INNER JOIN

INNER JOIN mantém somente linhas que possuem correspondência nas duas tabelas.

```
SELECT
    q.numero,
    t.nome_tipo
FROM quarto AS q
INNER JOIN tipo_quarto AS t
    ON q.id_tipo = t.id_tipo;
```

A cláusula ON define como as tabelas se relacionam.

8. LEFT JOIN

LEFT JOIN preserva todas as linhas da tabela à esquerda.

```
SELECT
    h.home,
    r.id_reserva
FROM hospede AS h
LEFT JOIN reserva AS r
    ON h.id_hospede = r.id_hospede;
```

Hóspedes sem reserva continuam aparecendo, com dados de reserva como NULL.

Pergunta útil:

Qual conjunto precisa obrigatoriamente aparecer completo?

Se a resposta for “todos os hóspedes”, hospede deve estar do lado preservado da junção.

9. RIGHT JOIN

RIGHT JOIN preserva todas as linhas da tabela à direita.

```
FROM quarto AS q
RIGHT JOIN tipo_quarto AS t
    ON q.id_tipo = t.id_tipo;
```

Uma consulta com RIGHT JOIN pode frequentemente ser reescrita invertendo as tabelas e usando LEFT JOIN.

10. Encontrando registros sem correspondência

Uma aplicação importante de LEFT JOIN é localizar registros sem relacionamento.

Exemplo:

```
SELECT
    h.id_hospede,
    h.nome
FROM hospede AS h
```

```
LEFT JOIN reserva AS r
  ON h.id_hospede = r.id_hospede;
WHERE r.id_reserva IS NULL;
```

Essa consulta encontra hóspedes que nunca fizeram reserva.

11. Múltiplos JOIN

Podemos combinar diversas tabelas:

```
SELECT
  h.nome,
  q.numero,
  t.nome_tipo,
  r.data_checkin
FROM reserva AS r
INNER JOIN hospede AS h
  ON r.id_hospede = h.id_hospede
INNER JOIN quarto AS q
  ON r.id_quarto = q.id_quarto
INNER JOIN tipo_quarto AS t
  ON q.id_tipo = t.id_tipo;
```

Visualize o caminho:

```
hospede
|
reserva
|
quarto
|
tipo_quarto
```

12. ON e WHERE em junções externas

Em LEFT JOIN, posicionar um filtro no ON ou no WHERE pode alterar o resultado.

Por exemplo:

```
FROM hospede AS h
LEFT JOIN reserva AS r
  ON h.id_hospede = r.id_hospede
  AND r.status = 'Confirmada'
```

Preserva todos os hóspedes.

Já:

```
WHERE r.status = 'Confirmada'
```

pode remover hóspedes sem reservas confirmadas.

Exercícios – Parte II

Exercício 11

Liste o número de cada quarto e o nome de seu tipo.
Mostre apenas quartos que possuam um tipo cadastrado.

Resposta: SELECT q.numero, t.nome_tipo
FROM quarto AS q
INNER JOIN tipo_quarto AS t ON q.id_tipo = t.id_tipo;

Exercício 12

Para cada reserva, mostre:

- hóspede;
- número do quarto;
- data de check-in;
- data de check-out.

Use INNER JOIN.

Resposta:

```
SELECT
    h.nome AS hospede,
    q.numero AS quarto,
    r.data_checkin,
    r.data_checkout
FROM reserva AS r
INNER JOIN hospede AS h ON r.id_hospede = h.id_hospede
INNER JOIN quarto AS q ON r.id_quarto = q.id_quarto;
```

Exercício 13

Liste as reservas realizadas em quartos do tipo Luxo.
Apresente:

- hóspede;
- quarto;
- tipo;
- valor_total.

Resposta:

```
SELECT
    h.nome AS hospede,
    q.numero AS quarto,
    t.nome_tipo AS tipo,
    r.valor_total
FROM reserva AS r
INNER JOIN hospede AS h ON r.id_hospede = h.id_hospede
```

```
INNER JOIN quarto AS q ON r.id_quarto = q.id_quarto
INNER JOIN tipo_quarto AS t ON q.id_tipo = t.id_tipo
WHERE t.nome_tipo = 'Luxo';
```

Exercício 14

Liste todos os hóspedes e, quando houver, suas reservas.
Hóspedes sem reserva também devem aparecer.

Resposta: SELECT h.nome, r.id_reserva, r.status
FROM hospede AS h
LEFT JOIN reserva AS r ON h.id_hospede = r.id_hospede;

Exercício 15

Liste todos os quartos e suas reservas.
Quartos nunca reservados devem permanecer no resultado.

Resposta: SELECT q.numero, r.id_reserva, r.status
FROM quarto AS q
LEFT JOIN reserva AS r ON q.id_quarto = r.id_quarto;

Exercício 16

Liste apenas hóspedes que nunca realizaram uma reserva.
Use LEFT JOIN e IS NULL .

Resposta: SELECT h.id_hospede, h.nome
FROM hospede AS h
LEFT JOIN reserva AS r ON h.id_hospede = r.id_hospede
WHERE r.id_reserva IS NULL;

Exercício 17

Liste todos os tipos de quarto e os quartos associados.
Tipos que não possuem nenhum quarto cadastrado também devem aparecer.

Resposta: SELECT t.nome_tipo, q.numero
FROM tipo_quarto AS t
LEFT JOIN quarto AS q ON t.id_tipo = q.id_tipo;

Exercício 18

Liste apenas quartos que nunca foram utilizados em nenhuma reserva.
Apresente:

- id;
- número;
- andar;
- status.

Resposta: SELECT q.id_quarto, q.numero, q.andar, q.status
FROM quarto AS q

LEFT JOIN reserva AS r ON q.id_quarto = r.id_quarto
WHERE r.id_reserva IS NULL;

Exercício 19

Liste todos os tipos de quarto e os quartos associados.
Tipos sem quartos também devem aparecer.
Resolva primeiro com LEFT JOIN.
Depois reescreva utilizando RIGHT JOIN.

Resposta:

```
-- Versão com LEFT JOIN
SELECT t.nome_tipo, q.numero
FROM tipo_quarto AS t
LEFT JOIN quarto AS q ON t.id_tipo = q.id_tipo;

-- Versão reescrita com RIGHT JOIN
SELECT t.nome_tipo, q.numero
FROM quarto AS q
RIGHT JOIN tipo_quarto AS t ON q.id_tipo = t.id_tipo;
```

Exercício 20

Crie um relatório contendo

- hóspede;
- número do quarto;
- tipo do quarto;
- check-in;
- check-out;
- status da reserva.

Todos os hóspedes devem aparecer, inclusive os que nunca reservaram um quarto.

Resposta:

```
SELECT
  h.nome AS hospede,
  q.numero AS quarto,
  t.nome_tipo AS tipo_quarto,
  r.data_checkin,
  r.data_checkout,
  r.status AS status_reserva
FROM hospede AS h
LEFT JOIN reserva AS r ON h.id_hospede = r.id_hospede
LEFT JOIN quarto AS q ON r.id_quarto = q.id_quarto
LEFT JOIN tipo_quarto AS t ON q.id_tipo = t.id_tipo;
```

Parte III – Aliases, DISTINCT e expressões calculadas

13. Aliases

Aliases dão nomes temporários.

Para tabelas:

```
FROM hospede AS h
```

Para colunas:

```
SELECT h.nome AS hospede
```

Eles ajudam quando:

- há várias tabelas;
- nomes são longos.
- expressões calculadas precisam de nome;
- a mesma tabela aparece mais de uma vez.

14. DISTINCT

Para eliminar valores repetidos:

```
SELECT DISTINCT cidade  
FROM hospede;
```

Use DISTINCT quando a duplicidade não fizer sentido para o resultado.

15. Colunas calculadas

No PostgreSQL, a diferença entre duas datas retorna a quantidade de dias.

Assim, podemos calcular a quantidade de diárias:

```
SELECT  
    id_reserva,  
    data_checkout - data_checkin AS quantidade_diarias  
FROM reserva;
```

Também podemos calcular um valor estimado:

```
(data_checkout - data_checkin) * t.valor_diaria
```

16. Autorrelacionamento

Na tabela funcionario, um supervisor também é um funcionário.

Por isso, podemos utilizar a mesma tabela duas vezes:

```
SELECT  
    f.nome AS funcionario,  
    s.nome AS supervisor  
FROM funcionario AS f  
INNER JOIN funcionario AS s  
    ON f.id_supervisor = s.id_funcionario;
```

Exercícios – Parte III

Exercício 21

Liste todas as cidades dos hóspedes sem valores duplicados.

Resposta: SELECT DISTINCT cidade
FROM hospede
WHERE cidade IS NOT NULL;

Exercício 22

Para cada tipo de quarto, apresente:

- nome;
- valor da diária;
- valor estimado de uma estadia de 7 dias.

Dê à coluna calculada o alias:
valor_semana

Resposta:

```
SELECT
    nome_tipo,
    valor_diaria,
    (valor_diaria * 7) AS valor_semana
FROM tipo_quarto;
```

Exercício 23

Para cada reserva, mostre:

- hóspede;
- número do quarto;
- quantidade de diárias;
- valor total;
- valor médio por diária.

Utilize expressões calculadas e aliases.

Resposta:

```
SELECT
    h.nome AS hospede,
    q.numero AS quarto,
    (r.data_checkout - r.data_checkin) AS quantidade_diarias,
    r.valor_total,
    (r.valor_total / (r.data_checkout - r.data_checkin)) AS valor_medio_diaria
FROM reserva AS r
INNER JOIN hospede AS h ON r.id_hospede = h.id_hospede
INNER JOIN quarto AS q ON r.id_quarto = q.id_quarto;
```

Exercício 24

Utilize os aliases h, r e q para consultar:

- nome do hóspede;

- número do quarto;
- valor total.

Mostre apenas reservas com valor superior a 1000.00.

Resposta:

```
SELECT
    h.nome AS hospede,
    q.numero AS quarto,
    r.valor_total
FROM reserva AS r
INNER JOIN hospede AS h ON r.id_hospede = h.id_hospede
INNER JOIN quarto AS q ON r.id_quarto = q.id_quarto
WHERE r.valor_total > 1000.00;
```

Exercício 25

Liste os estados dos hóspedes sem valores duplicados.

Depois explique por que DISTINCT altera a quantidade de linhas do resultado.

Resposta: SELECT DISTINCT estado
FROM hospede

O DISTINCT altera a quantidade de linhas do resultado porque consolida ocorrências repetidas em um único registro. Por exemplo, se vários hóspedes forem do estado 'PR', a consulta sem DISTINCT retornará o estado de cada um (várias linhas), enquanto com DISTINCT retornará a sigla 'PR' apenas uma vez.

Exercício 26

Utilizando a tabela funcionario, apresente:

- nome do funcionário;
- cargo;
- nome do supervisor.

Use a tabela funcionario duas vezes.

Resposta:

```
SELECT
    f.nome AS funcionario,
    f.cargo,
    s.nome AS supervisor
FROM funcionario AS f
LEFT JOIN funcionario AS s ON f.id_supervisor = s.id_funcionario;
```

Parte IV – Agrupamento e funções de agregação

17. Funções de agregação

As principais funções utilizadas são:
COUNT → quantidade

SUM → soma
AVG → média

Exemplo:
SELECT COUNT(*)
FROM reserva;

18. GROUP BY

Para resumir reservas por hóspede:

```
SELECT
    id_hospede,
    COUNT(*) AS quantidade
FROM reserva
GROUP BY id_hospede;
```

Cada hóspede forma um grupo.

Uma regra prática:

Colunas do SELECT que não participam de funções de agregação normalmente precisam aparecer no GROUP BY.

19. COUNT e LEFT JOIN

Em consultas com LEFT JOIN, existe diferença entre:

COUNT(*)

e:

COUNT(r.id_reserva)

Para um hóspede sem reservas, COUNT(r.id_reserva) retorna 0, pois valores NULL não são contados.

20. HAVING

WHERE filtra linhas antes do agrupamento.

HAVING filtra grupos depois da agregação.

```
SELECT
    id_hospede,
    COUNT(*) AS quantidade
FROM reserva
GROUP BY id_hospede
HAVING COUNT(*) >= 2;
```

21. ORDER BY

ORDER BY organiza o resultado:

ORDER BY quantidade DESC

Podemos combinar critérios:

ORDER BY quantidade DESC, nome ASC

Exercícios – Parte IV

Exercício 27

Conte quantas reservas existem no banco.

Resposta: SELECT COUNT(*) AS total_reservas
FROM reserva;

Exercício 28

Mostre a quantidade de reservas realizadas por cada hóspede que possui pelo menos uma reserva.

Resposta: SELECT id_hospede, COUNT(*) AS quantidade_reservas
FROM reserva
GROUP BY id_hospede;

Exercício 29

Mostre todos os hóspedes e a quantidade de reservas de cada um.
Hóspedes sem reservas devem aparecer com quantidade 0.

Resposta:
SELECT
 h.nome,
 COUNT(r.id_reserva) AS quantidade_reservas
FROM hospede AS h
LEFT JOIN reserva AS r ON h.id_hospede = r.id_hospede
GROUP BY h.id_hospede, h.nome;

Exercício 30

Para cada tipo de quarto, mostre:

- nome do tipo;
- quantidade de reservas realizadas em quartos desse tipo.

Resposta:
SELECT
 t.nome_tipo,
 COUNT(r.id_reserva) AS quantidade_reservas
FROM tipo_quarto AS t
LEFT JOIN quarto AS q ON t.id_tipo = q.id_tipo
LEFT JOIN reserva AS r ON q.id_quarto = r.id_quarto
GROUP BY t.id_tipo, t.nome_tipo;

Exercício 31

Para cada hóspede que possui reservas, mostre:

- nome;
- quantidade de reservas;
- total gasto.

Resposta:

```
SELECT
    h.nome,
    COUNT(r.id_reserva) AS quantidade_reservas,
    SUM(r.valor_total) AS total_gasto
FROM hospede AS h
INNER JOIN reserva AS r ON h.id_hospede = r.id_hospede
GROUP BY h.id_hospede, h.nome;
```

Exercício 32

Para cada hóspede, mostre:

- nome;
- quantidade de reservas;
- total gasto;
- média de valor das reservas.

Inclua hóspedes sem reservas.

Ordene pelo total gasto, do maior para o menor.

Resposta:

```
SELECT
    h.nome,
    COUNT(r.id_reserva) AS quantidade_reservas,
    COALESCE(SUM(r.valor_total), 0.00) AS total_gasto,
    COALESCE(AVG(r.valor_total), 0.00) AS media_valor_reservas
FROM hospede AS h
LEFT JOIN reserva AS r ON h.id_hospede = r.id_hospede
GROUP BY h.id_hospede, h.nome
ORDER BY total_gasto DESC;
```

Exercício 33

Liste apenas hóspedes que possuem duas ou mais reservas.

Use:

- JOIN;
- GROUP BY;
- COUNT;
- HAVING.

Resposta:

```
SELECT
    h.nome,
    COUNT(r.id_reserva) AS quantidade_reservas
```

```
FROM hospede AS h
INNER JOIN reserva AS r ON h.id_hospede = r.id_hospede
GROUP BY h.id_hospede, h.nome
HAVING COUNT(r.id_reserva) >= 2;
```

Exercício 34

Liste os tipos de quarto que aparecem em pelo menos duas reservas.

Apresente:

- tipo;
- quantidade de reservas;
- total faturado;
- média do valor das reservas.

Ordene da maior para a menor quantidade.

Resposta:

```
SELECT
    t.nome_tipo,
    COUNT(r.id_reserva) AS quantidade_reservas,
    SUM(r.valor_total) AS total_faturado,
    AVG(r.valor_total) AS media_valor
FROM tipo_quarto AS t
INNER JOIN quarto AS q ON t.id_tipo = q.id_tipo
INNER JOIN reserva AS r ON q.id_quarto = r.id_quarto
GROUP BY t.id_tipo, t.nome_tipo
HAVING COUNT(r.id_reserva) >= 2
ORDER BY quantidade_reservas DESC;
```

Exercício 35

Para cada tipo de quarto, apresente:

- nome do tipo;
- quantidade de quartos cadastrados;
- quantidade total de reservas feitas nesses quartos.

Tipos sem quartos também devem aparecer.

Evite contar o mesmo quarto várias vezes devido às reservas.

Resposta:

```
SELECT
    t.nome_tipo,
    COUNT(DISTINCT q.id_quarto) AS quantidade_quartos,
    COUNT(r.id_reserva) AS total_reservas
FROM tipo_quarto AS t
LEFT JOIN quarto AS q ON t.id_tipo = q.id_tipo
LEFT JOIN reserva AS r ON q.id_quarto = r.id_quarto
GROUP BY t.id_tipo, t.nome_tipo;
```

Parte V – Operações de conjunto e subconsultas

22. UNION e UNION ALL

UNION combina resultados e elimina duplicatas.

```
SELECT...  
UNION  
SELECT...
```

UNION ALL preserva duplicatas.

As consultas devem retornar:

- a mesma quantidade de colunas;
- tipos compatíveis.

23. INTERSECT

INTERSECT retorna valores presentes nas duas consultas.

Exemplo de pergunta:

Quais hóspedes já reservaram tantos quartos Standard quantos quartos Luxo?

24. Subconsultas

Uma subconsulta pode funcionar como uma tabela temporária:

```
SELECT ...  
FROM (  
    SELECT ...  
    FROM reserva  
    WHERE ...  
) AS reservas_filtradas;
```

Também pode aparecer em condições:

```
WHERE id_hospede IN (  
    SELECT ...  
)
```

ou:

```
WHERE id_hospede NOT IN (  
    SELECT ...  
)
```

Exercícios – Parte V

Exercício 36

Crie uma única lista contendo:

- cidades de hóspedes que reservaram quartos Standard;
- cidades de hóspedes que reservaram quartos Luxo.

Primeiro utilize UNION ALL.

Depois utilize UNION e compare os resultados.

Resposta:

```
-- Com UNION ALL (mantém duplicatas)
SELECT h.cidade
FROM hospede AS h
INNER JOIN reserva AS r ON h.id_hospede = r.id_hospede
INNER JOIN quarto AS q ON r.id_quarto = q.id_quarto
INNER JOIN tipo_quarto AS t ON q.id_tipo = t.id_tipo
WHERE t.nome_tipo = 'Standard'
UNION ALL
SELECT h.cidade
FROM hospede AS h
INNER JOIN reserva AS r ON h.id_hospede = r.id_hospede
INNER JOIN quarto AS q ON r.id_quarto = q.id_quarto
INNER JOIN tipo_quarto AS t ON q.id_tipo = t.id_tipo
WHERE t.nome_tipo = 'Luxo';
```

```
-- Com UNION (remove duplicatas)
SELECT h.cidade
FROM hospede AS h
INNER JOIN reserva AS r ON h.id_hospede = r.id_hospede
INNER JOIN quarto AS q ON r.id_quarto = q.id_quarto
INNER JOIN tipo_quarto AS t ON q.id_tipo = t.id_tipo
WHERE t.nome_tipo = 'Standard'
UNION
SELECT h.cidade
FROM hospede AS h
INNER JOIN reserva AS r ON h.id_hospede = r.id_hospede
INNER JOIN quarto AS q ON r.id_quarto = q.id_quarto
INNER JOIN tipo_quarto AS t ON q.id_tipo = t.id_tipo
WHERE t.nome_tipo = 'Luxo';
```

Exercício 37

Crie uma lista única com os nomes de hóspedes que possuem reservas Concluída e os hóspedes que possuem reservas Confirmada.

Elimine duplicatas com UNION.

Resposta:

```
SELECT h.nome
FROM hospede AS h
INNER JOIN reserva AS r ON h.id_hospede = r.id_hospede
WHERE r.status = 'Concluída'
UNION
SELECT h.nome
FROM hospede AS h
INNER JOIN reserva AS r ON h.id_hospede = r.id_hospede
WHERE r.status = 'Confirmada';
```

Exercício 38

Liste hóspedes que já realizaram reserva em quarto Standard e também em quarto Luxo. Utilize INTERSECT.

Resposta:

```
SELECT h.id_hospede, h.nome
FROM hospede AS h
INNER JOIN reserva AS r ON h.id_hospede = r.id_hospede
INNER JOIN quarto AS q ON r.id_quarto = q.id_quarto
INNER JOIN tipo_quarto AS t ON q.id_tipo = t.id_tipo
WHERE t.nome_tipo = 'Standard'
INTERSECT
SELECT h.id_hospede, h.nome
FROM hospede AS h
INNER JOIN reserva AS r ON h.id_hospede = r.id_hospede
INNER JOIN quarto AS q ON r.id_quarto = q.id_quarto
INNER JOIN tipo_quarto AS t ON q.id_tipo = t.id_tipo
WHERE t.nome_tipo = 'Luxo';
```

Exercício 39

Crie uma subconsulta no FROM que selecione reservas cujo valor seja igual ou superior a 1000.00.

Utilize o resultado para mostrar:

- hóspede;
- número do quarto;
- valor total.

Resposta:

```
SELECT
    h.nome AS hospede,
    q.numero AS quarto,
    res.valor_total
FROM (
    SELECT id_hospede, id_quarto, valor_total
    FROM reserva
    WHERE valor_total >= 1000.00
) AS res
INNER JOIN hospede AS h ON res.id_hospede = h.id_hospede
INNER JOIN quarto AS q ON res.id_quarto = q.id_quarto;
```

Exercício 40

Liste os hóspedes que realizaram pelo menos uma reserva em quarto do tipo Luxo.

Resolva utilizando uma subconsulta.

Cada hóspede deve aparecer apenas uma vez.

Resposta:

```

SELECT id_hospede, nome
FROM hospede
WHERE id_hospede IN (
    SELECT r.id_hospede
    FROM reserva AS r
    INNER JOIN quarto AS q ON r.id_quarto = q.id_quarto
    INNER JOIN tipo_quarto AS t ON q.id_tipo = t.id_tipo
    WHERE t.nome_tipo = 'Luxo'
);

```

Exercício 41

Liste hóspedes que **nunca** reservaram um quarto do tipo Luxo.
 Utilize uma subconsulta com NOT IN ou uma estratégia equivalente.

Resposta:

```

SELECT id_hospede, nome
FROM hospede
WHERE id_hospede NOT IN (
    SELECT r.id_hospede
    FROM reserva AS r
    INNER JOIN quarto AS q ON r.id_quarto = q.id_quarto
    INNER JOIN tipo_quarto AS t ON q.id_tipo = t.id_tipo
    WHERE t.nome_tipo = 'Luxo'
);

```

Exercício 42

Calcule o total gasto por hóspede.

Depois utilize essa consulta como subconsulta para listar somente hóspedes cujo total gasto seja superior à média dos totais gastos pelos hóspedes que possuem reservas

Resposta:

```

SELECT
    totais.id_hospede,
    totais.nome,
    totais.total_gasto
FROM (
    SELECT
        h.id_hospede,
        h.nome,
        SUM(r.valor_total) AS total_gasto
    FROM hospede AS h
    INNER JOIN reserva AS r ON h.id_hospede = r.id_hospede
    GROUP BY h.id_hospede, h.nome
) AS totais
WHERE totais.total_gasto > (
    SELECT AVG(soma_hospede)
    FROM (
        SELECT SUM(valor_total) AS soma_hospede

```

```
FROM reserva
GROUP BY id_hospede
) AS medias
);
```

Parte VI – CRUD com PostgreSQL

25. O conceito de CRUD

CRUD representa:

Create → INSERT

Read → SELECT

Update → UPDATE

Delete → DELETE

Exemplo de cadastro:

```
INSERT INTO hospede
```

```
(id_hospede, nome, email, cidade, estado, telefone)
```

```
VALUES
```

```
(20, 'Marcos Silva', 'marcos@email.com', 'Curitiba', 'PR', '41999990020');
```

Consulta:

```
SELECT *
```

```
FROM hospede
```

```
WHERE id_hospede = 20;
```

Atualização:

```
UPDATE hospede
```

```
SET cidade = 'Londrina'
```

```
WHERE id_hospede = 20;
```

Exclusão:

```
DELETE FROM hospede
```

```
WHERE id_hospede = 20;
```

26. INSERT

Uma forma recomendada é informar as colunas explicitamente.

Também é possível inserir várias linhas:

```
INSERT INTO hospede
```

```
(id_hospede, nome, email, cidade, estado, telefone)
```

```
VALUES
```

```
(...),
```

```
(...),
```

```
(...);
```

27. UPDATE

Estrutura:

```
UPDATE tabela
```

```
SET coluna = valor
```

```
WHERE condição;
```

Mais de uma coluna:

```
UPDATE hospede
```

```
SET
```

```
    cidade = 'Curitiba',
```

```
    estado = 'PR'
```

```
WHERE id_hospede = 20;
```

Com expressão:

```
UPDATE tipo_quarto
```

```
SET valor_diaria = valor_diaria * 1.08
```

```
WHERE valor_diaria < 400;
```

Cuidado

Sem WHERE, todas as linhas podem ser atualizadas.

Antes:

```
SELECT *
```

```
FROM tipo_quarto
```

```
WHERE valor_diaria < 400;
```

Depois, execute o UPDATE.

28. DELETE

Estrutura:

```
DELETE FROM tabela
```

```
WHERE condição;
```

Uma estratégia segura é usar primeiro:

```
SELECT *
```

```
FROM tabela
```

```
WHERE condição;
```

e só depois realizar a exclusão.

29. Integridade referencial

Uma reserva referência:

hospede

quarto

Portanto, o PostgreSQL não deve permitir uma reserva apontando para um hóspede ou quarto inexistente.

Dá mesma forma, um hóspede com reservas associadas não pode ser excluído diretamente enquanto as reservas existirem, considerando as restrições definidas neste banco.

Exercícios – Parte VI

Exercício 43

Cadastre um novo hóspede no PostgreSQL.

Depois utilize SELECT para confirmar o cadastro.

Resposta:

-- Inserção

```
INSERT INTO hospede (id_hospede, nome, email, cidade, estado, telefone)
VALUES (11, 'Ricardo Santos', 'ricardo@email.com', 'Curitiba', 'PR', '41999990011');
```

-- Confirmação

```
SELECT * FROM hospede WHERE id_hospede = 11;
```

Exercício 44

Cadastre três novos quartos utilizando um único INSERT.

Utilize tipos de quarto já existentes.

Depois consulte somente os registros recém-inseridos.

Resposta:

-- Inserção em lote

```
INSERT INTO quarto (id_quarto, numero, andar, status, vista, id_tipo)
VALUES
(601, '601', 6, 'Disponível', 'Mar', 3),
(602, '602', 6, 'Disponível', 'Cidade', 1),
(603, '603', 6, 'Manutenção', NULL, 2);
```

-- Confirmação

```
SELECT * FROM quarto WHERE id_quarto IN (601, 602, 603);
```

Exercício 45

Escolha um hóspede de teste e execute:

1. SELECT antes da alteração;
2. UPDATE da cidade;
3. SELECT depois da alteração.

Resposta:

-- 1. Consulta antes

```
SELECT * FROM hospede WHERE id_hospede = 1;
```

-- 2. Atualização

```
UPDATE hospede  
SET cidade = 'Ponta Grossa'  
WHERE id_hospede = 1;
```

-- 3. Consulta depois

```
SELECT * FROM hospede WHERE id_hospede = 1;
```

Exercício 46

Aumente em 10% o valor da diária de todos os tipos de quarto cujo valor atual seja inferior a 400.00.

Antes do UPDATE, execute um SELECT com a mesma condição

Resposta:

-- Consulta antes

```
SELECT * FROM tipo_quarto WHERE valor_diaria < 400.00;
```

-- Atualização

```
UPDATE tipo_quarto  
SET valor_diaria = valor_diaria * 1.10  
WHERE valor_diaria < 400.00;
```

Exercício 47

Cadastre um hóspede temporário.

Depois:

1. consulte;
2. exclua com DELETE;
3. consulte novamente para confirmar a remoção.

Resposta:

-- Inserção temporária

```
INSERT INTO hospede (id_hospede, nome, email, cidade, estado, telefone)  
VALUES (99, 'Hóspede Teste', 'teste@email.com', 'Teste', 'TS', '00000000000')
```

-- 1. Consulta antes da exclusão

```
SELECT * FROM hospede WHERE id_hospede = 99;
```

-- 2. Exclusão

```
DELETE FROM hospede WHERE id_hospede = 99;
```

-- 3. Consulta para confirmar a remoção

```
SELECT * FROM hospede WHERE id_hospede = 99;
```

Exercício 48

Tente cadastrar uma reserva utilizando um id_hospede inexistente.

Registre:

1. o comando executado;

2. o erro produzido pelo PostgreSQL;
3. por que a chave estrangeira impede a operação.

Resposta: INSERT INTO reserva

(id_reserva, id_hospede, id_quarto, data_checkin, data_checkout, quantidade_hospedes, status, valor_total)

VALUES

(100, 999, 101, '2026-10-10', '2026-10-12', 1, 'Confirmada', 360.00);

Erro retornado pelo PostgreSQL: ERRO: inserção ou atualização em tabela "reserva" viola restrição de chave estrangeira "fk_reserva_hospede"

DETALHE: Chave (id_hospede)=(999) não está presente na tabela "hospede".

Motivo do bloqueio: A restrição de integridade referencial (FOREIGN KEY) garante a consistência do banco de dados. Ela impede que um registro em uma tabela filha (reserva) faça referência a uma chave primária inexistente na tabela pai (hospede), evitando que existam reservas órfãs associadas a hóspedes não cadastrados.